Reading Binary Digits from an Image with Projection Profiles and a Width Rule

Nithin D H
Indian Institute of Technology Kanpur (IIT-Kanpur)
Kanpur, India

Abstract

Not every recognition problem needs a learned model. We read a 138-digit binary number printed as six lines of light text on a dark background, using only classical image processing. The image is binarised, text lines are found from the horizontal projection profile, characters are segmented with the vertical projection profile, and each character is classified by its width*: in this typeface a '1' is a narrow stroke 10–12 px wide and a '0' a wide oval 21–22 px wide. The width threshold is chosen automatically at the largest gap in the observed widths, so no pixel constant is hard-coded. The program recovers all 138 bits (64 ones, 74 zeros), which we verified line by line against the image. We discuss when such a rule-based reader is the right tool and what would break it.*

1. Introduction

The task. The input (Fig. 1) is a 631×345 image containing six lines of 23 binary digits each. The goal is to output the digit string and its decimal value.

Key observation. With only two symbols, all that is needed is one feature that separates them. In this typeface the '1' is a thin vertical stroke and the '0' a wide oval, so the width of a character alone decides its class, with a 9 px empty gap between the two width clusters. A trained model would add complexity without adding accuracy.

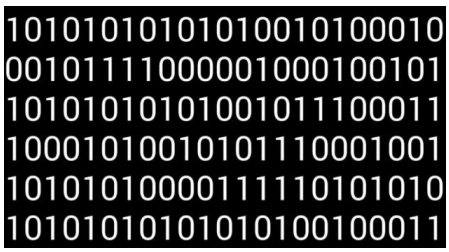


Figure 1. Input image: six lines of light '0'/'1' digits on a dark background.

2. Related Work

Character segmentation by projection profiles is one of the oldest OCR techniques [1] and remains part of modern engines such as Tesseract [4]. Learned recognisers [2] become necessary when the glyph classes cannot be separated by simple geometric features.

3. Method

Figure 3 shows the flow of the program.

1. **Binarise.** Convert to grayscale and threshold at 128, so text pixels are 1.
2. **Crop.** Restrict to the bounding box of all text pixels.
3. **Lines (horizontal projection).** Sum text pixels per row; rows whose sum exceeds 3% of the width belong to a text line. Runs of such rows give six lines.
4. **Characters (vertical projection).** Within each line, sum text pixels per column; columns above 8% of the line height belong to a character. Runs of such columns give each digit's horizontal extent.
5. **Automatic width threshold.** Sort the distinct character widths, find the largest gap between consecutive values, and place the threshold at its midpoint. This adapts to the image's resolution.
6. **Classify and assemble.** A width below the threshold gives '1', otherwise '0'. Lines are concatenated from top to bottom.

4. Results and Discussion

4.1. Segmentation and the width rule

Figure 2 shows the intermediate signals. The row-sum profile has six clean plateaus, and the column-sum profile of each line separates the digits with zero-valued gaps. The 138 measured widths fall into two tight clusters, 10–12 px (64 characters) and 21–22 px (74 characters), and the automatic threshold lands at 16.5 px, in the middle of the empty gap.

4.2. Output

Table 1 gives the decoded lines. Concatenated, the 138-bit string has the decimal value

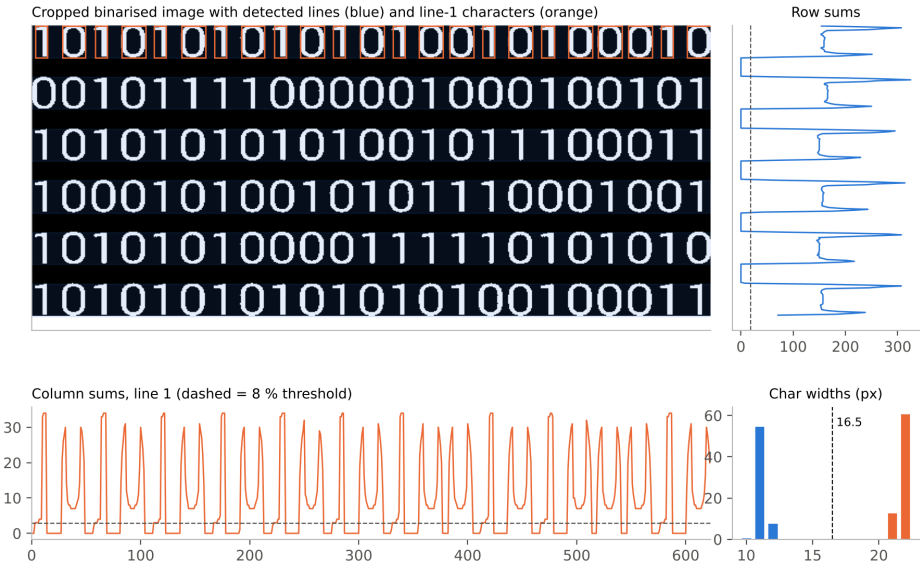


Figure 2. Top: detected text lines (blue bands) and the character boxes of line 1 (orange), with the row-sum profile on the right. Bottom: column-sum profile of line 1 and the histogram of all 138 character widths, showing two well-separated clusters.

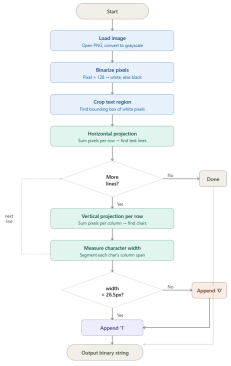


Figure 3. Flowchart of the approach.

Table 1. Decoded digits, 23 per line.

Line	Bits
1	10101010101010010100010
2	00101111000001000100101
3	10101010101001011100011
4	10001010010101110001001
5	10101010000111110101010
6	10101010101010100100011

232291987565042107861626583843617067324707.

Every line was checked by eye against the image; re-running the program reproduces the output exactly.

Why width and not ink density? A natural first feature is the fraction of text pixels inside each character box. In this font the thin ‘1’ and the hollow ‘0’ have similar ink-to-box ratios, so density does not separate them reliably. Width is a structural property of the glyph shapes and does not depend on stroke thickness.

5. Conclusion & Future Work

Projection profiles plus a self-calibrating width threshold decode the full 138-bit number with no training data and no errors.

Limitations. The method assumes axis-aligned, non-touching characters, a uniform high-contrast background, and a typeface in which ‘1’ is much narrower than ‘0’.

Extensions. Otsu thresholding [3] would handle uneven lighting; deskewing and connected-component analysis would handle rotated or touching glyphs; and a small template matcher or CNN [2] on the segmented glyphs would generalise to typefaces where width is not decisive.

Acknowledgement. This work was carried out as part of EE655 Computer Vision & Deep Learning at IIT Kanpur under the supervision of Dr. Koteswar Rao Jerripothula.

References

[1] R. G. Casey and E. Lecolinet. A survey of methods and strategies in character segmentation. *IEEE Trans. Pattern Anal. Mach. Intell.*, 18(7):690–706, 1996.

[2] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proc. IEEE*, 86(11):2278–2324, 1998.

[3] N. Otsu. A threshold selection method from gray-level histograms. *IEEE Trans. Systems, Man, and Cybernetics*, 9(1):62–66, 1979.

[4] R. Smith. An overview of the Tesseract OCR engine. In *Int. Conf. Document Analysis and Recognition*, 2007.